Assignment 7

Support Vector Classifiers and Machines

Author: Samuel Fredric Berg

Student ID: sb224sc

Date: 2026-03-13

Course: Machine Learning 4DT905

Conceptual

1. **SVMs** goal is to create a hyperplane which seperates classes which tries to maximize the margin to the closest point in each class. If a hyperpalane cannot be created, either a slack variable is introduced or a dimensionality increase is preformed, which makes it easier to make a hyperplane.

2. Generally **SVMs** are robust to overfitting, however noisy data can cause overfitting. Certain kernels such as the *RBF* kernal can be more precise than *linear* kernals, but they are also more prone to overfitting. Small C values allows for more misclassifications, which reduces the risk of overfitting.

Practical

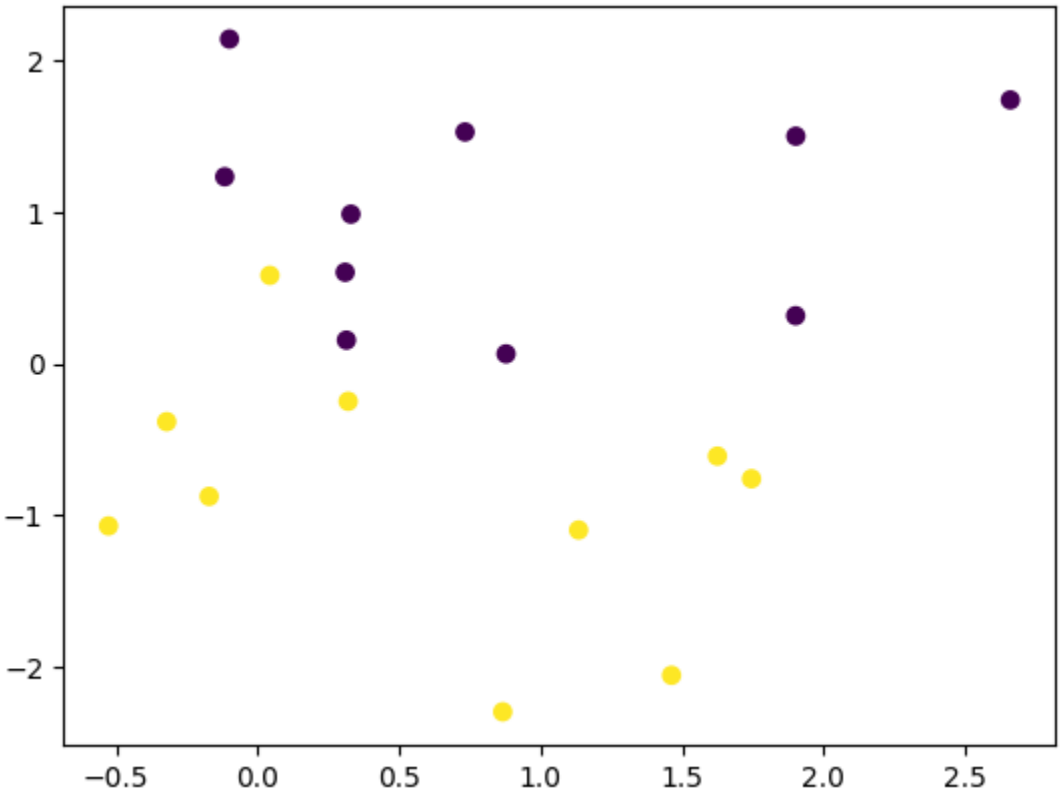
Imports

In [47]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import confusion_matrix

import warnings

warnings.filterwarnings("ignore", category=UserWarning, module="sklearn")
```

Step 1: Generate data and get an overview

In [48]:
```
np.random.seed(1)
X = np.random.randn(20, 2)
Y = np.concatenate([-np.ones(10), np.ones(10)])
```

```
X[Y == 1] += 1

plt.scatter(X[:, 0], X[:, 1], c=(3 - Y))
plt.show()

print(X)
print(Y)
```

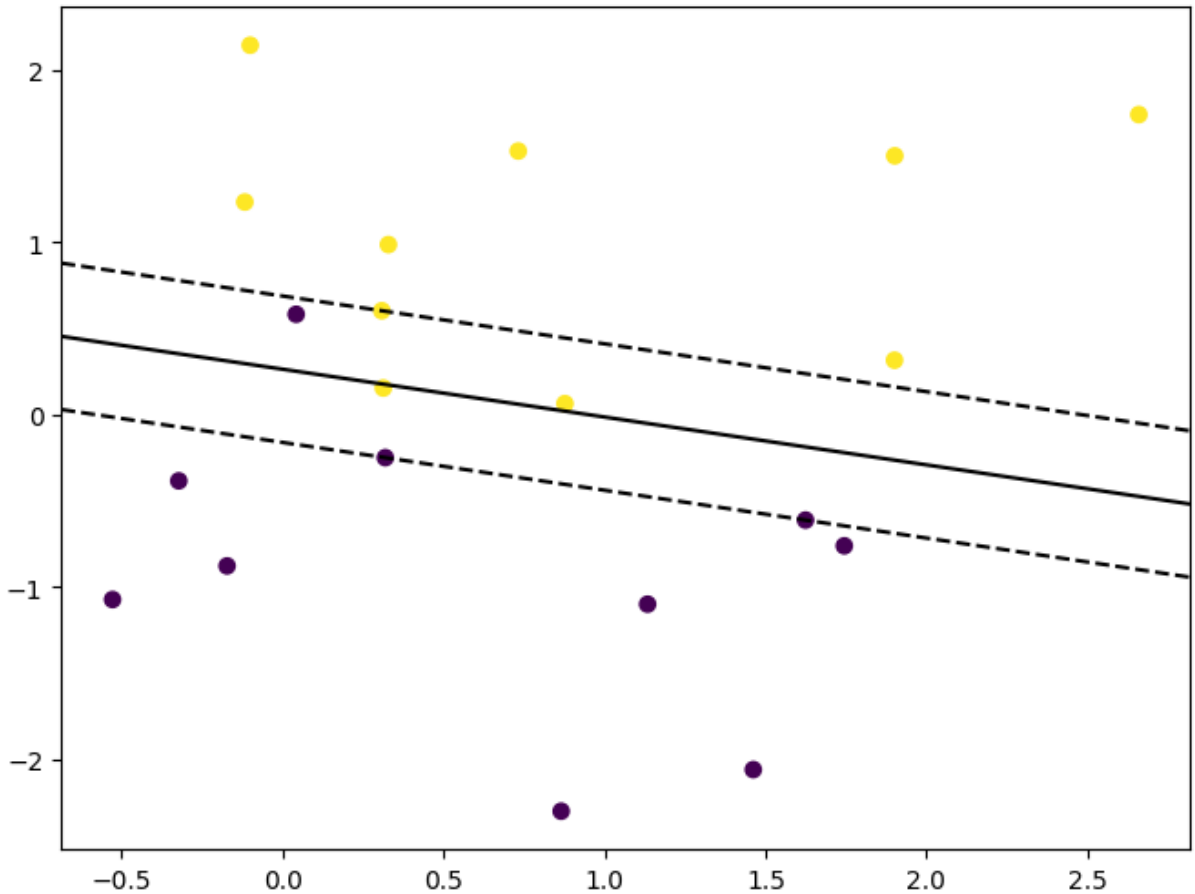



```
[[ 1.62434536 -0.61175641]
 [-0.52817175 -1.07296862]
 [ 0.86540763 -2.3015387 ]
 [ 1.74481176 -0.7612069 ]
 [ 0.3190391  -0.24937038]
 [ 1.46210794 -2.06014071]
 [-0.3224172  -0.38405435]
 [ 1.13376944 -1.09989127]
 [-0.17242821 -0.87785842]
 [ 0.04221375  0.58281521]
 [-0.10061918  2.14472371]
 [ 1.90159072  1.50249434]
 [ 1.90085595  0.31627214]
 [ 0.87710977  0.06423057]
 [ 0.73211192  1.53035547]
 [ 0.30833925  0.60324647]
 [ 0.3128273   0.15479436]
 [ 0.32875387  0.9873354 ]
 [-0.11731035  1.2344157 ]
 [ 2.65980218  1.74204416]]
[-1. -1. -1. -1. -1. -1. -1. -1. -1. -1.  1.  1.  1.  1.  1.  1.  1.  1.
  1.  1.]
```

In [49]:
```
df = pd.DataFrame({"X1": X[:, 0], "X2": X[:, 1], "Y": Y.astype(int)})
```

```python
print(df)

svm = SVC(kernel="linear", C=10)
svm.fit(df[["X1", "X2"]], df["Y"])

plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=Y)

ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()

XX, YY = np.meshgrid(
    np.linspace(xlim[0], xlim[1], 50), np.linspace(ylim[0], ylim[1], 50)
)
XY = np.c_[XX.ravel(), YY.ravel()]
Z = svm.decision_function(XY).reshape(XX.shape)

ax.contour(
    XX,
    YY,
    Z,
    levels=[-1, 0, 1],
    linestyles=["dashed", "solid", "dashed"],
    colors="black",
)

plt.show()
```

```
          X1        X2  Y
0   1.624345 -0.611756 -1
1  -0.528172 -1.072969 -1
2   0.865408 -2.301539 -1
3   1.744812 -0.761207 -1
4   0.319039 -0.249370 -1
5   1.462108 -2.060141 -1
6  -0.322417 -0.384054 -1
7   1.133769 -1.099891 -1
8  -0.172428 -0.877858 -1
9   0.042214  0.582815 -1
10 -0.100619  2.144724  1
11  1.901591  1.502494  1
12  1.900856  0.316272  1
13  0.877110  0.064231  1
14  0.732112  1.530355  1
15  0.308339  0.603246  1
16  0.312827  0.154794  1
17  0.328754  0.987335  1
18 -0.117310  1.234416  1
19  2.659802  1.742044  1
```

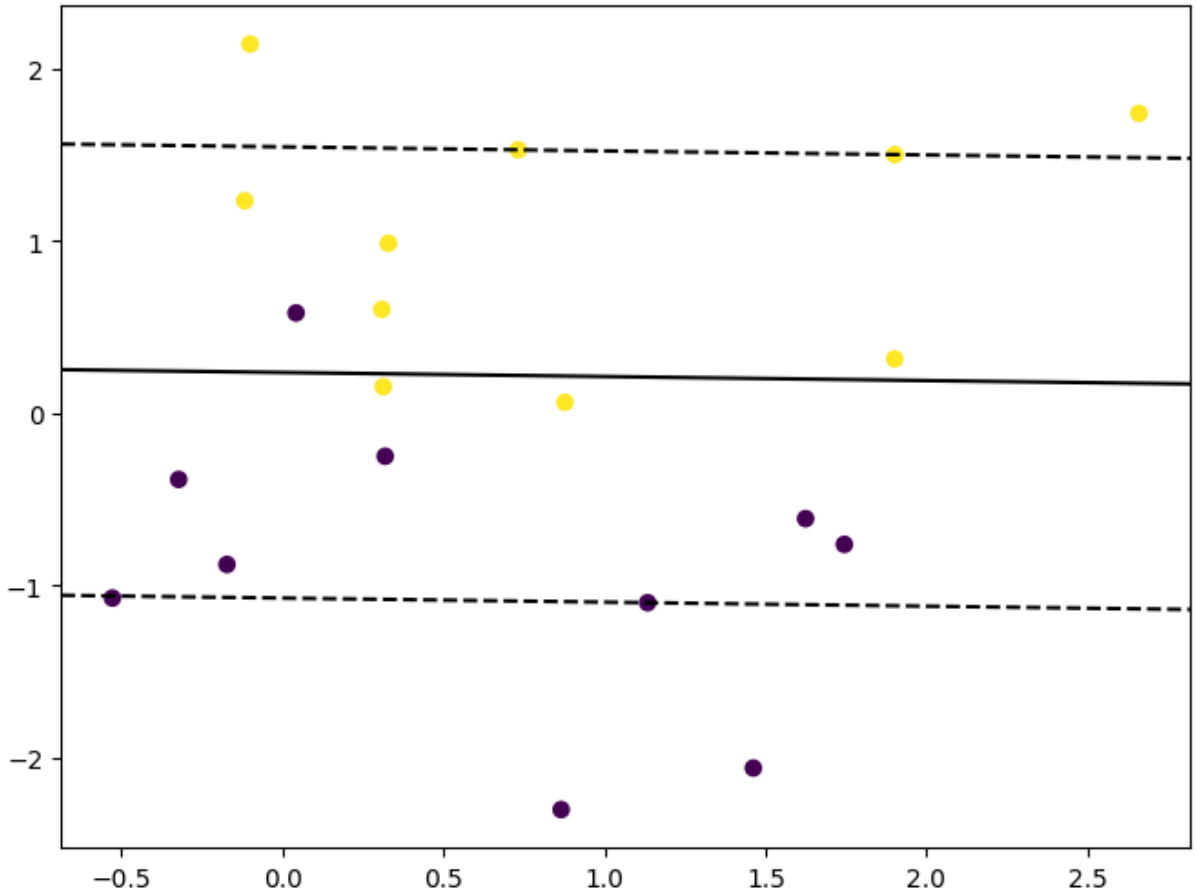


In [50]:
```
support_vector_indices = svm.support_
print(support_vector_indices)
```

```
[ 0  4  9 13 15 16]
```

In [51]:
```
svm = SVC(kernel="linear", C=0.1)
svm.fit(df[["X1", "X2"]], df["Y"])

plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=Y)

ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()

XX, YY = np.meshgrid(
    np.linspace(xlim[0], xlim[1], 50), np.linspace(ylim[0], ylim[1], 50)
)
XY = np.c_[XX.ravel(), YY.ravel()]
Z = svm.decision_function(XY).reshape(XX.shape)

ax.contour(
    XX,
    YY,
    Z,
    levels=[-1, 0, 1],
    linestyles=["dashed", "solid", "dashed"],
    colors="black",
```

```
)

plt.show()
```



Interpretation:

Comparing the two linear SVMs, the larger $C = 10$ produces a **narrow margin** that tries hard to correctly classify all training points. The smaller $C = 0.1$ allows for a **wider margin** and tolerates more misclassifications on the training data, which corresponds to higher bias but lower variance. The wider margin in the $C = 0.1$ model acts as a regularizer and reduces the risk of overfitting to the training data.

In [52]:
```
param_grid = {"C": [0.001, 0.01, 0.1, 1, 5, 10, 100]}

grid_search = GridSearchCV(SVC(kernel="linear"), param_grid, cv=10)
grid_search.fit(df[["X1", "X2"]], df["Y"])

cv_results = pd.DataFrame(grid_search.cv_results_)[
    ["param_C", "mean_test_score", "std_test_score"]
]
print(cv_results)

print("\nBest C:", grid_search.best_params_["C"])
print("Best Accuracy:", grid_search.best_score_)
```

```
   param_C  mean_test score  std_test_score
0    0.001             0.95        0.150000
1    0.010             0.95        0.150000
2    0.100             0.85        0.229129
3    1.000             0.90        0.200000
4    5.000             0.90        0.200000
5   10.000             0.90        0.200000
6  100.000             0.90        0.200000

Best C: 0.001
Best Accuracy: 0.95
```

Interpretation:

Get equal results for $C = 0.01$ and $C = 0.001$ which gives a accuracy of 95%.

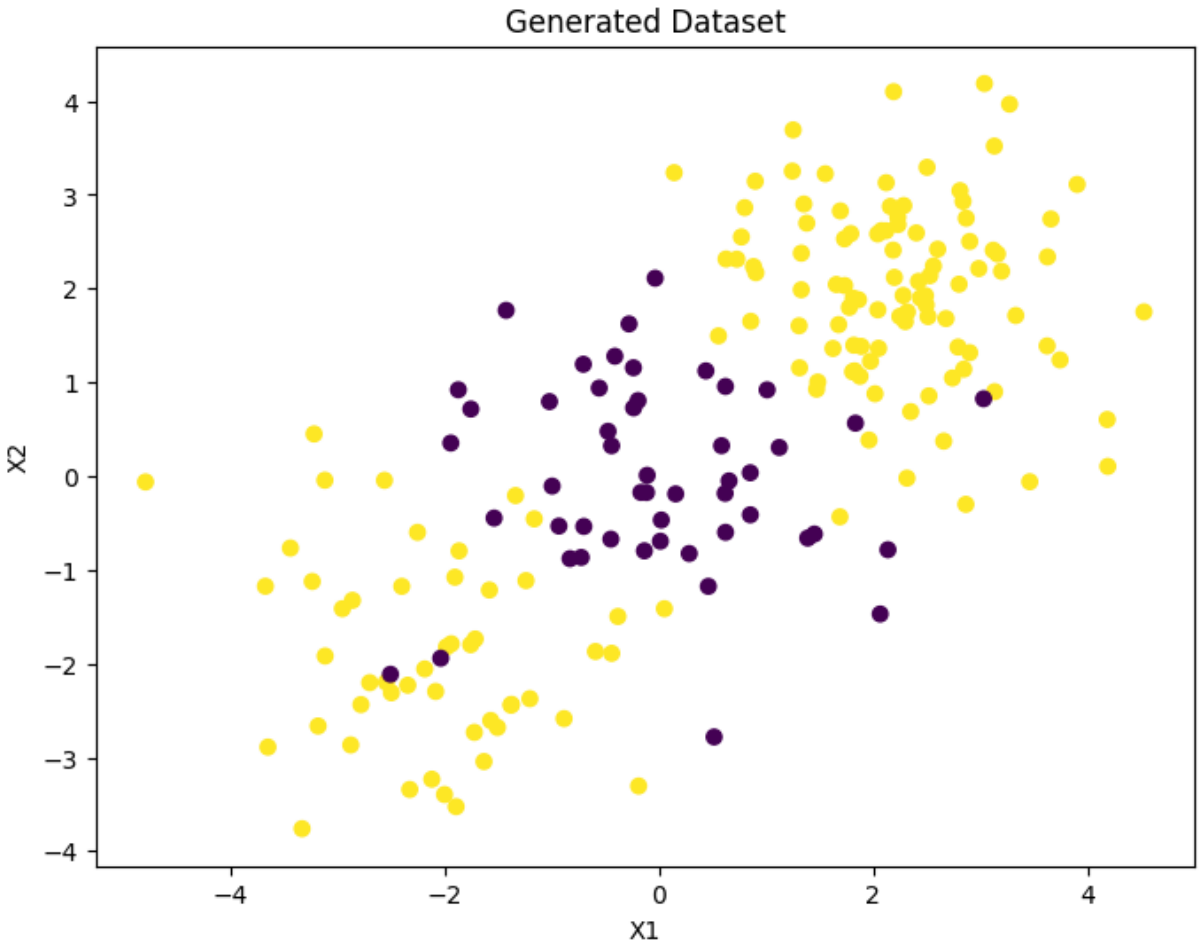
In [53]:
```
np.random.seed(1)
svm = SVC(kernel="linear", C=0.001)
svm.fit(df[["X1", "X2"]], df["Y"])
X_test = np.random.randn(20, 2)
Y_test = np.random.choice([-1, 1], size=20, replace=True)
X_test[Y_test == 1] += 1

Y_pred = svm.predict(X_test)
conf_matrix = confusion_matrix(Y_test, Y_pred)
print("Confusion Matrix:\n", conf_matrix)
```

```
Confusion Matrix:
 [[7 4]
 [1 8]]
```

Interpretation:

With the newly generated data the accuracy is calculated to 75%. Read from the confusion matrix.

In [54]:
```
np.random.seed(1)
X = np.random.randn(200, 2)
X[:100] += 2
X[100:150] -= 2
Y = np.concatenate([-np.ones(150), np.ones(50)])
df = pd.DataFrame({"X1": X[:, 0], "X2": X[:, 1], "Y": Y.astype(int)})

plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=(3 - Y), cmap="viridis")
plt.xlabel("X1")
plt.ylabel("X2")
plt.title("Generated Dataset")
plt.show()

print(X)
print(Y)
```


Generated Dataset
X2
X1
4
3
2
1
0
−1
−2
−3
−4
−4
−2
0
2
4

```
[[ 3.62434536  1.38824359]
 [ 1.47182825  0.92703138]
 [ 2.86540763 -0.3015387 ]
 [ 3.74481176  1.2387931 ]
 [ 2.3190391   1.75062962]
 [ 3.46210794 -0.06014071]
 [ 1.6775828   1.61594565]
 [ 3.13376944  0.90010873]
 [ 1.82757179  1.12214158]
 [ 2.04221375  2.58281521]
 [ 0.89938082  3.14472371]
 [ 2.90159072  2.50249434]
 [ 2.90085595  1.31627214]
 [ 1.87710977  1.06423057]
 [ 1.73211192  2.53035547]
 [ 1.30833925  1.60324647]
 [ 1.3128273   1.15479436]
 [ 1.32875387  1.9873354 ]
 [ 0.88268965  2.2344157 ]
 [ 3.65980218  2.74204416]
 [ 1.80816445  1.11237104]
 [ 1.25284171  3.6924546 ]
 [ 2.05080775  1.36300435]
 [ 2.19091548  4.10025514]
 [ 2.12015895  2.61720311]
 [ 2.30017032  1.64775015]
 [ 0.8574818   1.65065728]
 [ 1.79110577  2.58662319]
 [ 2.83898341  2.93110208]
 [ 2.28558733  2.88514116]
 [ 1.24560206  3.25286816]
 [ 2.51292982  1.70190716]
 [ 2.48851815  1.92442829]
 [ 3.13162939  3.51981682]
 [ 4.18557541  0.60350366]
 [ 0.55588619  1.49553414]
 [ 2.16003707  2.87616892]
 [ 2.31563495 -0.02220122]
 [ 1.69379599  2.82797464]
 [ 2.23009474  2.76201118]
 [ 1.77767186  1.79924193]
 [ 2.18656139  2.41005165]
 [ 2.19829972  2.11900865]
 [ 1.32933771  2.37756379]
 [ 2.12182127  3.12948391]
 [ 3.19891788  2.18515642]
 [ 1.62471505  1.36126959]
 [ 2.42349435  2.07734007]
 [ 1.65614632  2.04359686]
 [ 1.37999916  2.69803203]
 [ 1.55287144  3.2245077 ]
 [ 2.40349164  2.59357852]
 [ 0.90508815  2.16938243]
 [ 2.74055645  1.0462994 ]
 [ 1.73378149  2.03261455]
 [ 0.62688268  2.31515939]
```

```
[ 2.84616065  1.14048406]
[ 2.35054598  0.68771659]
[ 1.96130449  0.38422765]
[ 3.12141771  2.40890054]
[ 1.97538304  1.22483838]
[ 3.27375593  3.96710175]
[ 0.14201814  3.23616403]
[ 3.62765075  2.3380117 ]
[ 0.80073197  2.86334532]
[ 1.8190797   1.39607937]
[ 0.76994186  2.5505375 ]
[ 2.79280687  1.37646927]
[ 2.52057634  0.85565861]
[ 2.80186103  2.0465673 ]
[ 1.81343023  1.89825413]
[ 2.86888616  2.75041164]
[ 2.52946532  2.13770121]
[ 2.07782113  2.61838026]
[ 2.23249456  2.68255141]
[ 1.68988323 -0.43483776]
[ 3.0388246   4.18697965]
[ 2.44136444  1.89984477]
[ 1.86355526  1.88094581]
[ 2.01740941  0.87798127]
[ 1.48290554  1.00297317]
[ 2.24879916  1.70335885]
[ 2.49521132  1.82529684]
[ 2.98633519  2.2135339 ]
[ 4.19069973  0.10363908]
[ 1.35308331  2.90148689]
[ 4.52832571  1.75136522]
[ 2.04366899  1.77368576]
[ 3.33145711  1.71269214]
[ 2.68006984  1.6801984 ]
[ 0.72744124  2.31354772]
[ 2.50318481  3.29322588]
[ 1.88955297  1.38263794]
[ 2.5627611   2.24073709]
[ 2.28066508  1.9268873 ]
[ 3.16033857  2.36949272]
[ 3.90465871  3.1110567 ]
[ 2.6590498   0.37256166]
[ 2.60231928  2.4202822 ]
[ 2.81095167  3.04444209]
[-2.40087819 -1.17599438]
[-2.56230543 -0.04512192]
[-3.33195167 -3.76068856]
[-3.65072127 -2.89055558]
[-3.1191154  -0.0439211 ]
[-2.3264995  -3.34267579]
[-0.88561702 -2.58652394]
[-3.23685338 -1.12416107]
[-1.37663782 -2.43495668]
[-0.59246    -1.87089842]
[-0.3830504  -1.49725912]
[-0.44119446 -1.8905973 ]
```

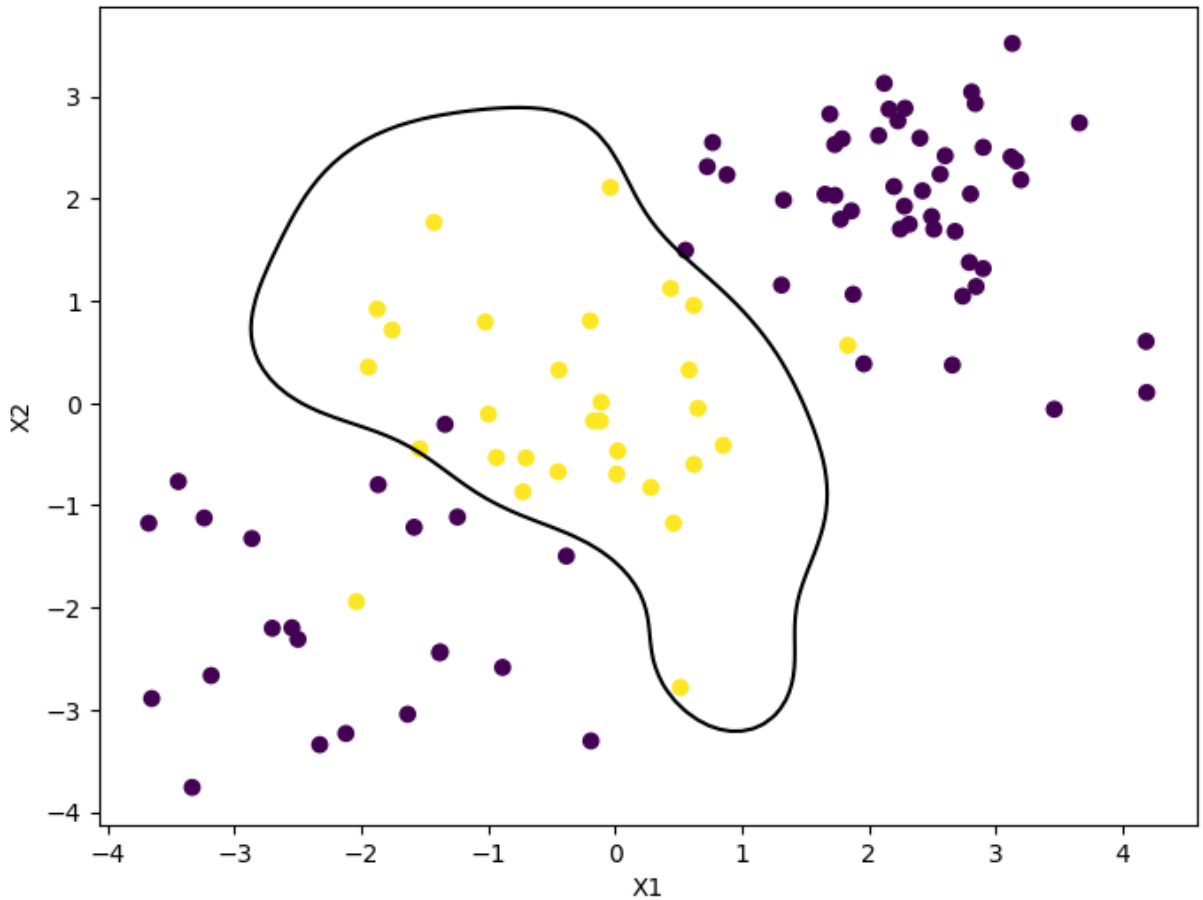
```
[-3.2197444   0.44936865]
[-2.54577417 -2.19883786]
[-2.7003985  -2.20339445]
[-1.75733056 -1.79816982]
[-1.33897971 -0.20784179]
[-2.12046457 -3.23312074]
[-3.18231813 -2.66575452]
[-3.67419581 -1.17497018]
[-2.49821356 -2.31098498]
[-2.00189148 -3.39662042]
[-2.86131636 -1.32528847]
[-1.38146087 -2.44317193]
[-0.18946509 -3.30572692]
[-2.34498721 -2.23083974]
[-4.793085   -0.06247119]
[-1.63366799 -3.04458938]
[ 0.05117344 -1.414338  ]
[-1.57047386 -2.6069984 ]
[-1.89377728 -3.52568032]
[-1.20497391 -2.37443832]
[-1.8659518  -0.79794514]
[-1.71525189 -1.73753255]
[-1.7235007  -2.7332716 ]
[-1.16399528 -0.45664089]
[-1.24119434 -1.11509119]
[-2.87728152 -2.86778722]
[-3.44087602 -0.76774693]
[-2.25417987 -0.60015606]
[-2.78191168 -2.43750898]
[-1.90457491 -1.07854993]
[-1.9392498  -1.78887524]
[-1.98347243 -1.82281228]
[-3.11647002 -1.9190729 ]
[-2.18657899 -2.05682448]
[-1.50766344 -2.68067814]
[-2.08450803 -2.29736188]
[-1.582698   -1.21522935]
[-2.95542526 -1.41408957]
[ 2.06578332 -1.47115693]
[-0.8301719  -0.8805776 ]
[-0.27909772  1.62284909]
[ 0.01335268 -0.6946936 ]
[ 0.6218035  -0.59980453]
[ 1.12341216  0.30526704]
[ 1.3887794  -0.66134424]
[ 3.03085711  0.82458463]
[ 0.65458015 -0.05118845]
[-0.72559712 -0.86776868]
[-0.13597733 -0.79726979]
[ 0.28267571 -0.82609743]
[ 0.6210827   0.9561217 ]
[-0.70584051  1.19268607]
[-0.23794194  1.15528789]
[ 0.43816635  1.12232832]
[-0.9970198  -0.10679399]
[ 1.45142926 -0.61803685]
```

```
 [-2.03720123 -1.94258918]
 [-2.50644065 -2.11416392]
 [-0.41163916  1.27852808]
 [-0.44222928  0.32352735]
 [-0.10999149  0.00854895]
 [-0.16819884 -0.17418034]
 [ 0.4611641  -1.17598267]
 [ 1.01012718  0.92001793]
 [-0.19505734  0.80539342]
 [-0.70134443 -0.53722302]
 [ 0.15626385 -0.19022103]
 [-0.44873803 -0.67244804]
 [-0.55749472  0.93916874]
 [-1.94332341  0.35249436]
 [-0.23643695  0.7278135 ]
 [ 0.51507361 -2.78253447]
 [ 0.58464661  0.32427424]
 [ 0.02186284 -0.46867382]
 [ 0.85328122 -0.41302931]
 [ 1.83471763  0.56438286]
 [ 2.13782807 -0.785534  ]
 [-1.75592564  0.7147896 ]
 [ 0.85270406  0.0353601 ]
 [-1.53879325 -0.44789518]
 [ 0.61798553 -0.18417633]
 [-0.11598519 -0.17545897]
 [-0.93391466 -0.53302033]
 [-1.42655542  1.76795995]
 [-0.47537288  0.47761018]
 [-1.02188594  0.79452824]
 [-1.87316098  0.92061512]
 [-0.03536792  2.11060505]]
[-1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1.
 -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1.
 -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1.
 -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1.
 -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1.
 -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1.
 -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1.
 -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1. -1.
 -1. -1. -1. -1. -1. -1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.
  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.
  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.
  1.  1.]
```

In [55]:

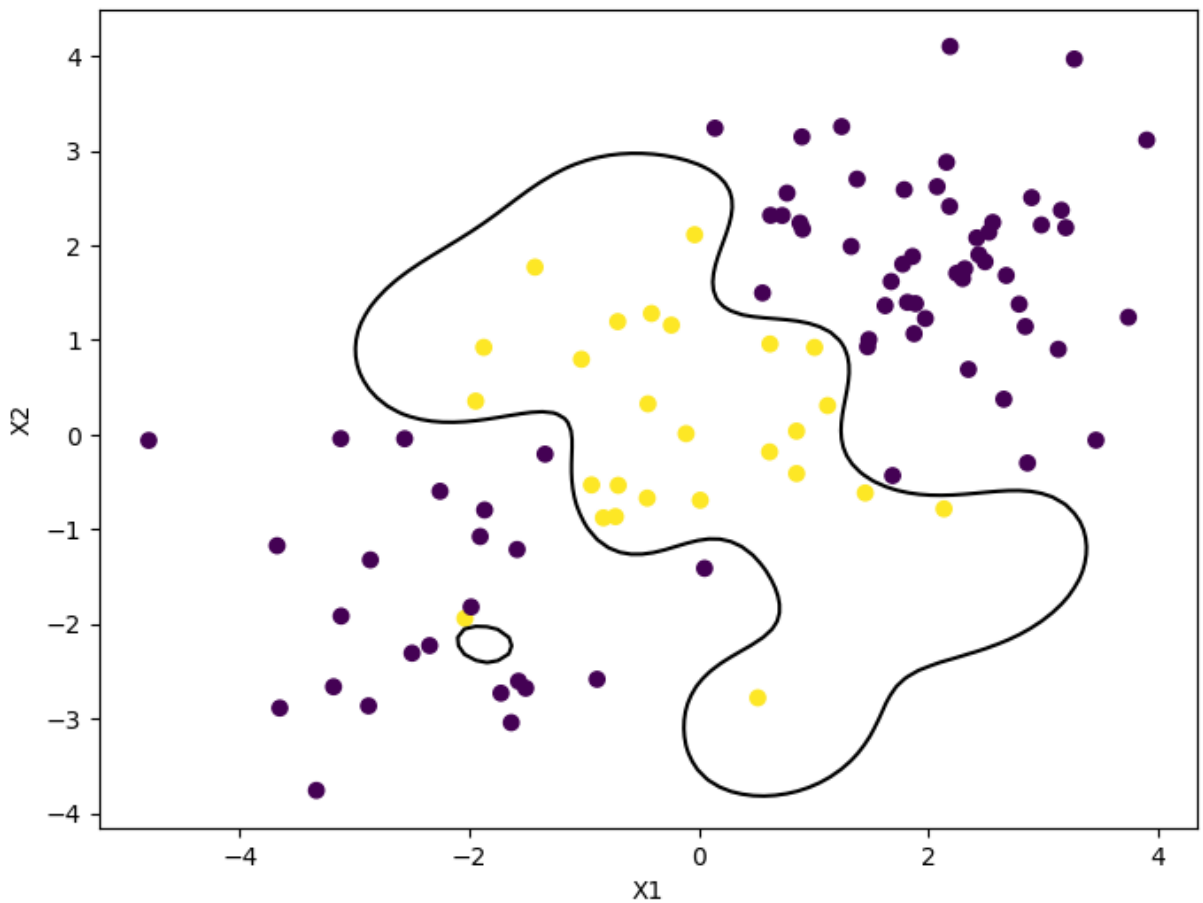
```
np.random.seed(1)
train_indices = np.random.choice(200, 100, replace=False)

train_data = df.iloc[train_indices]
test_data = df.drop(train_indices)

svmfit = SVC(kernel="rbf", gamma=1, C=1)
svmfit.fit(train_data[["X1", "X2"]], train_data["Y"])

plt.figure(figsize=(8, 6))
plt.scatter(train_data["X1"], train_data["X2"], c=train_data["Y"])
```

```
xlim, ylim = plt.xlim(), plt.ylim()
XX, YY = np.meshgrid(
    np.linspace(xlim[0], xlim[1], 100), np.linspace(ylim[0], ylim[1], 100)
)
XY = np.c_[XX.ravel(), YY.ravel()]
Z = svmfit.decision_function(XY).reshape(XX.shape)

plt.contour(XX, YY, Z, levels=[0], linestyles=["solid"], colors="black")

plt.xlabel("X1")
plt.ylabel("X2")
plt.show()
```



In [56]:
```
train_indices = np.random.choice(200, 100, replace=False)

train_data = df.iloc[train_indices]
test_data = df.drop(train_indices)

svmfit = SVC(kernel="rbf", gamma=1, C=100)
svmfit.fit(train_data[["X1", "X2"]], train_data["Y"])

plt.figure(figsize=(8, 6))
plt.scatter(train_data["X1"], train_data["X2"], c=train_data["Y"])
xlim, ylim = plt.xlim(), plt.ylim()
XX, YY = np.meshgrid(
    np.linspace(xlim[0], xlim[1], 100), np.linspace(ylim[0], ylim[1], 100)
)
XY = np.c_[XX.ravel(), YY.ravel()]
```

```
Z = svmfit.decision_function(XY).reshape(XX.shape)

plt.contour(XX, YY, Z, levels=[0], linestyles=["solid"], colors="black")

plt.xlabel("X1")
plt.ylabel("X2")
plt.show()
```



In [57]:
```
param_grid = {"C": [0.1, 1, 10, 100, 1000], "gamma": [0.5, 1]}

grid_search = GridSearchCV(SVC(kernel="rbf"), param_grid, cv=10)
grid_search.fit(df[["X1", "X2"]], df["Y"])

cv_results = pd.DataFrame(grid_search.cv_results_)[
    ["param_C", "param_gamma", "mean_test_score", "std_test_score"]
]
print(cv_results)

print("\nBest C:", grid_search.best_params_["C"])
print("Best Gamma:", grid_search.best_params_["gamma"])
print(f"Best Accuracy: {grid_search.best_score_:.2f}")
```

```
   param_C  param_gamma  mean_test_score  std_test_score
0      0.1          0.5            0.905        0.065000
1      0.1          1.0            0.865        0.063443
2      1.0          0.5            0.940        0.043589
3      1.0          1.0            0.940        0.043589
4     10.0          0.5            0.920        0.045826
5     10.0          1.0            0.920        0.033166
6    100.0          0.5            0.900        0.038730
7    100.0          1.0            0.880        0.090000
8   1000.0          0.5            0.870        0.122882
9   1000.0          1.0            0.865        0.118427

Best C: 1
Best Gamma: 0.5
Best Accuracy: 0.94
```

Interpretation:

$C = 1$ with $\gamma = 1.0$ or $\gamma = 0.5$ results in the highest accuracy of 94%.

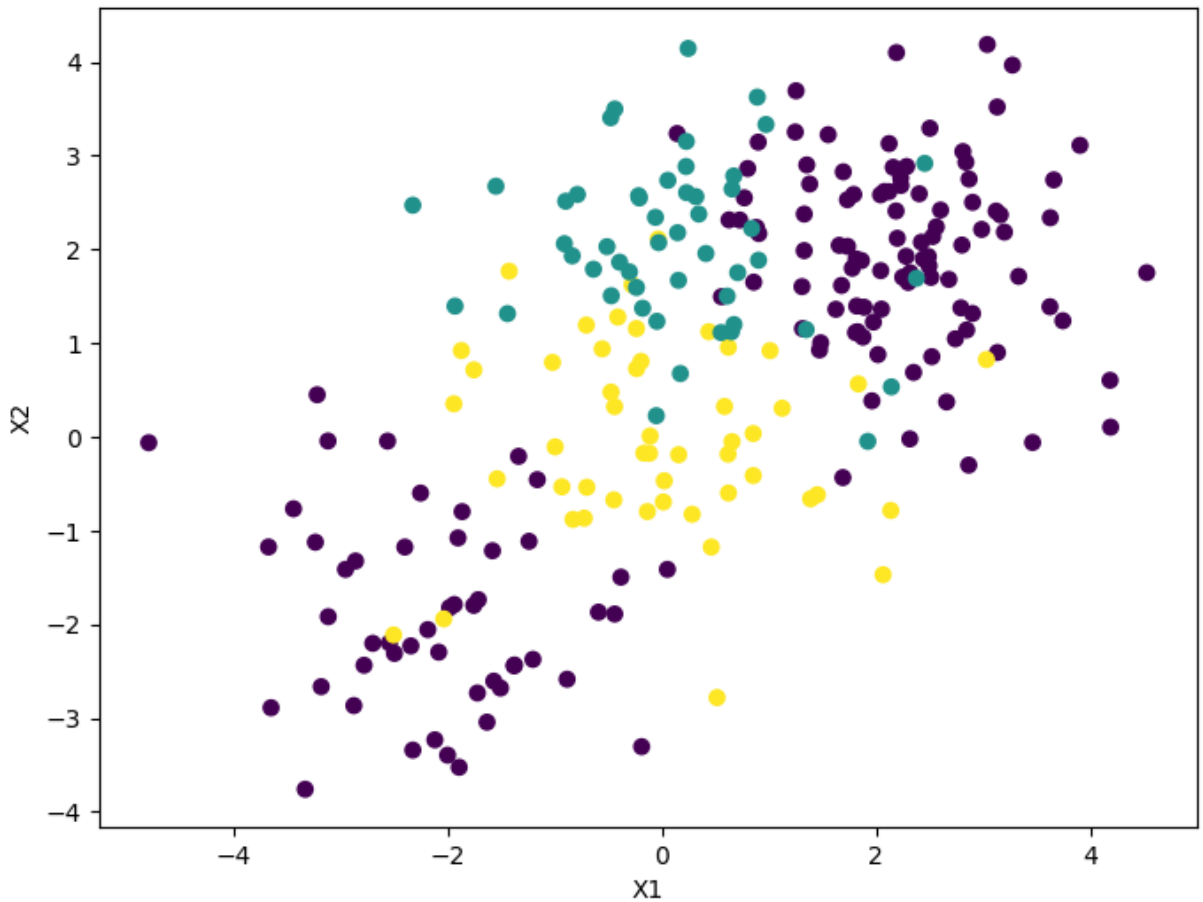
In [58]:
```
svm = SVC(kernel="rbf", C=1, gamma=0.5)
svm.fit(df[["X1", "X2"]], df["Y"])

X_test = np.random.randn(100, 2)
X_test[:50] += 2
X_test[50:75] -= 2
Y_test = np.concatenate([-np.ones(75), np.ones(25)])

Y_pred = svm.predict(X_test)
conf_matrix = confusion_matrix(Y_test, Y_pred)
print("Confusion Matrix:\n", conf_matrix)
```

```
Confusion Matrix:
 [[71  4]
 [ 3 22]]
```

Interpretation:

From the confusion matrix we get an accuracy of 93%.

In [60]:
```
X = np.vstack([X, np.random.randn(50, 2)])
Y = np.concatenate([Y, np.zeros(50)])

X[Y == 0, 1] += 2

df = pd.DataFrame({"X1": X[:, 0], "X2": X[:, 1], "Y": Y})

plt.figure(figsize=(8, 6))
plt.scatter(df["X1"], df["X2"], c=df["Y"])
plt.xlabel("X1")
plt.ylabel("X2")
plt.show()

print(df)
```

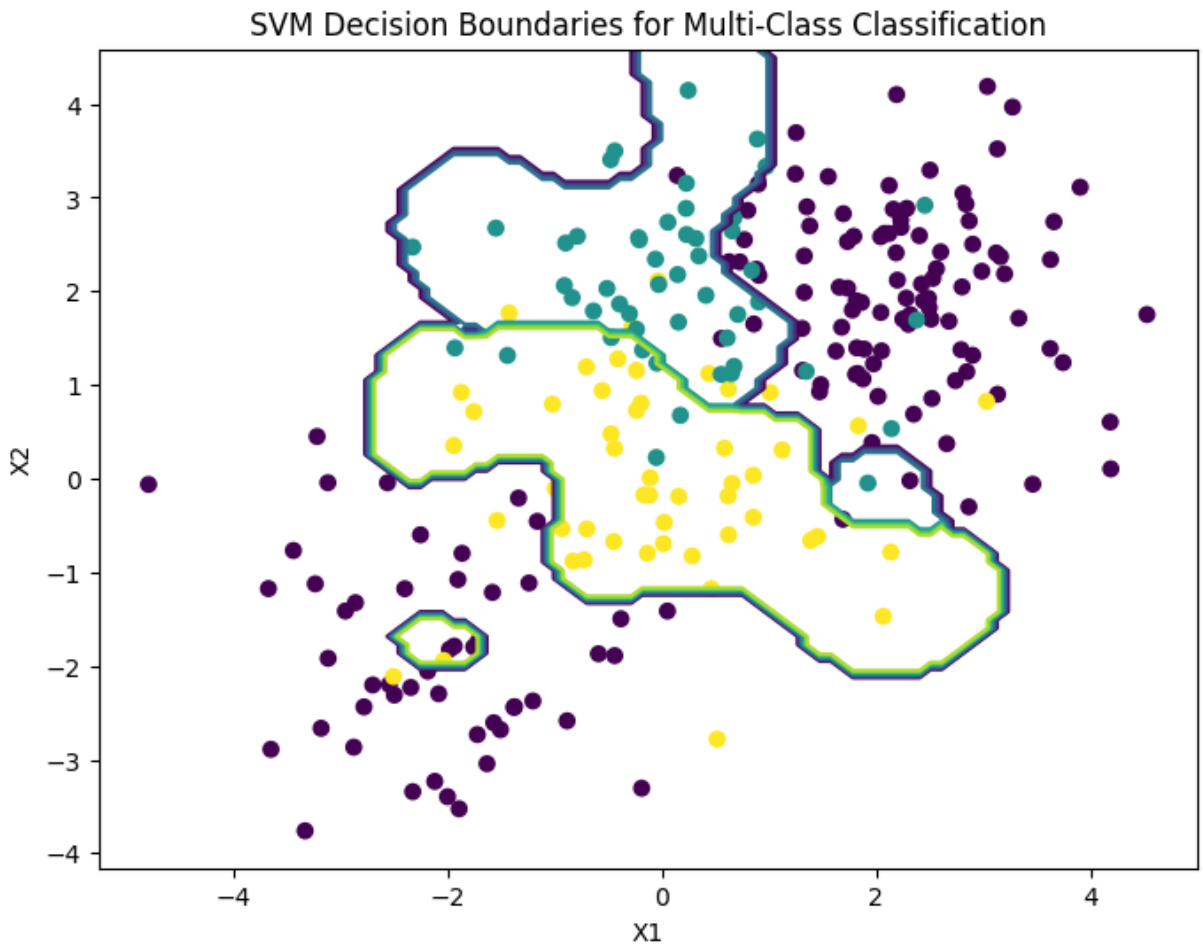



```
          X1        X2    Y
0    3.624345  1.388244 -1.0
1    1.471828  0.927031 -1.0
2    2.865408 -0.301539 -1.0
3    3.744812  1.238793 -1.0
4    2.319039  1.750630 -1.0
..        ...       ...  ...
245  1.347264  1.144185  0.0
246 -0.910924  2.060662  0.0
247  0.707985  1.752115  0.0
248 -0.472958  1.505882  0.0
249 -0.303401  1.760157  0.0

[250 rows x 3 columns]
```

In [61]:

```
np.random.seed(1)
train_indices = np.random.choice(250, 125, replace=False)

train_data = df.iloc[train_indices]
test_data = df.drop(train_indices)

svm = SVC(kernel="rbf", C=10, gamma=1)
svm.fit(train_data[["X1", "X2"]], train_data["Y"])

plt.figure(figsize=(8, 6))
plt.scatter(df["X1"], df["X2"], c=df["Y"])

xlim, ylim = plt.xlim(), plt.ylim()
XX, YY = np.meshgrid(
```

```
    np.linspace(xlim[0], xlim[1], 100), np.linspace(ylim[0], ylim[1], 100)
)

XY = np.c_[XX.ravel(), YY.ravel()]
Z = svm.predict(XY)
Z = Z.reshape(XX.shape)

plt.contour(XX, YY, Z)
plt.xlabel("X1")
plt.ylabel("X2")
plt.title("SVM Decision Boundaries for Multi-Class Classification")
plt.show()
```



In [62]:
```
param_grid = {"C": [0.1, 1, 10, 100, 1000], "gamma": [0.5, 1, 2, 3, 4]}

grid_search = GridSearchCV(SVC(kernel="rbf"), param_grid, cv=10)
grid_search.fit(df[["X1", "X2"]], df["Y"])

cv_results = pd.DataFrame(grid_search.cv_results_)[
    ["param_C", "param_gamma", "mean_test_score", "std_test_score"]
]
print(cv_results)

print("\nBest C:", grid_search.best_params_["C"])
print("Best Gamma:", grid_search.best_params_["gamma"])
print(f"Best Accuracy: {grid_search.best_score_:.2f}")
```

```
    param_C  param_gamma  mean_test_score  std_test_score
0       0.1          0.5            0.808    4.995998e-02
1       0.1          1.0            0.772    3.124100e-02
2       0.1          2.0            0.648    4.308132e-02
3       0.1          3.0            0.600    1.110223e-16
4       0.1          4.0            0.600    1.110223e-16
5       1.0          0.5            0.852    4.019950e-02
6       1.0          1.0            0.856    4.079216e-02
7       1.0          2.0            0.836    4.543127e-02
8       1.0          3.0            0.828    4.019950e-02
9       1.0          4.0            0.832    3.487119e-02
10     10.0          0.5            0.848    2.400000e-02
11     10.0          1.0            0.832    4.308132e-02
12     10.0          2.0            0.808    5.878775e-02
13     10.0          3.0            0.792    6.881860e-02
14     10.0          4.0            0.768    8.908423e-02
15    100.0          0.5            0.812    7.166589e-02
16    100.0          1.0            0.772    8.588364e-02
17    100.0          2.0            0.772    1.088853e-01
18    100.0          3.0            0.768    8.352245e-02
19    100.0          4.0            0.752    8.908423e-02
20   1000.0          0.5            0.760    9.797959e-02
21   1000.0          1.0            0.732    1.227029e-01
22   1000.0          2.0            0.756    9.372300e-02
23   1000.0          3.0            0.720    8.579044e-02
24   1000.0          4.0            0.704    8.039900e-02

Best C: 1
Best Gamma: 1
Best Accuracy: 0.86
```

In [63]:
```
svm = SVC(kernel="rbf", C=1, gamma=0.5)
svm.fit(train_data[["X1", "X2"]], train_data["Y"])
Y_pred = svm.predict(test_data[["X1", "X2"]])
cm = confusion_matrix(test_data["Y"], Y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[71  4]
 [ 3 22]]
```

Interpretation:

Get the same accuracy of 93%.

Load data

In [66]:
```
X_train = pd.read_csv("../data/Khan_xtrain.csv", index_col=0)
X_test = pd.read_csv("../data/Khan_xtest.csv", index_col=0)
Y_train = pd.read_csv("../data/Khan_ytrain.csv", index_col=0)
Y_test = pd.read_csv("../data/Khan_ytest.csv", index_col=0)

print(len(X_train), len(X_train.columns.tolist()))
print(len(X_test), len(X_test.columns.tolist()))
```

```
63 2308
20 2308
```

In [67]:
```
Y_train = Y_train.iloc[:, 0]

svm_model = SVC(kernel="linear", C=10)
svm_model.fit(X_train, Y_train)

print("Number of Support:", svm_model.n_support_)
print("Number of classes:", svm_model.classes_)
```

```
Number of Support: [ 7 18  9 20]
Number of classes: [1 2 3 4]
```

In [68]:
```
Y_train_pred = svm_model.predict(X_train)
conf_matrix = confusion_matrix(Y_train, Y_train_pred)
conf_matrix_df = pd.DataFrame(
    conf_matrix, index=svm_model.classes_, columns=svm_model.classes_
)

print(conf_matrix_df)
```

```
   1   2   3   4
1  8   0   0   0
2  0  23   0   0
3  0   0  12   0
4  0   0   0  20
```

In [69]:
```
Y_test_pred = svm_model.predict(X_test)
conf_matrix_test = confusion_matrix(Y_test, Y_test_pred)

conf_matrix_test_df = pd.DataFrame(
    conf_matrix_test, index=svm_model.classes_, columns=svm_model.classes_
)
print(conf_matrix_test_df)
```

```
   1  2  3  4
1  3  0  0  0
2  0  6  0  0
3  0  2  4  0
4  0  0  0  5
```

Interpretation:

Only two predictions are incorrect, which results in a accuracy of 90%.